langchain-mcp-adapters 완전 가이드

> **대상**: Python으로 MCP(Model Context Protocol) 클라이언트를 작성하려는 개발자 **기준 버전**: `langchain-mcp-adapters` v0.1.x 이상

목차 (Table of Contents)

1. langchain-mcp-adapters 개요
2. Transport 방식 비교: stdio vs Streamable HTTP
3. stdio 방식 상세 - 서버를 클라이언트에서 실행하는 이유
4. stdio 서버 실행 방식 - 서버 코드에 따른 command 설정
5. Streamable HTTP 방식 상세
6. stdio + HTTP 혼합 사용
7. Sync vs Async 방식
8. Session 관리: Stateless vs Stateful
    - 패턴 1: Stateful — 장기 세션 (프로그램 전체 유지)
    - 패턴 2: Stateful — 대화 이력 유지 (멀티턴 챗봇)
    - 패턴 3: Stateless — 요청마다 새 세션
    - 패턴 4: Stateless — MultiServerMCPClient
    - 패턴 5: Stateless — FastAPI 요청별 세션
    - 패턴 6: Stateful — FastAPI 앱 수명주기 전역 세션
    - 패턴 7: HTTP Transport Stateless
9. LangChain Agent와 결합하기
10. AI 답변 실시간 스트리밍
11. 여러 MCP 서버 동시 사용 시 주의사항
12. 전체 구조 요약

---

1. langchain-mcp-adapters 개요

`langchain-mcp-adapters`는 MCP(Model Context Protocol) 서버의 도구(Tool)를 **LangChain/LangGraph 에이전트**에서 바로 사용할 수 있도록 변환해주는 어댑터 라이브러리입니다.

핵심 역할

- MCP 서버와의 연결(Transport) 관리
- MCP `Tool` → LangChain `BaseTool` 자동 변환
- 단일/다중 서버 동시 연결 지원
- stdio, Streamable HTTP 두 가지 Transport 지원

설치

```
pip install langchain-mcp-adapters langchain-anthropic langgraph python-dotenv
```

패키지	역할
`langchain-mcp-adapters`	MCP ↔ LangChain 브릿지
`langchain-anthropic`	Claude LLM 연동
`langgraph`	ReAct 에이전트 (`create_react_agent`)
`mcp`	MCP 프로토콜 핵심 라이브러리 (자동 설치)
`python-dotenv`	`.env` 파일로 API 키 관리

주요 임포트 경로

```
# 단일 서버 — 세션 직접 관리
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from langchain_mcp_adapters.tools import load_mcp_tools

# 다중 서버 — 자동 세션 관리
from langchain_mcp_adapters.client import MultiServerMCPClient

# LangGraph 에이전트
from langgraph.prebuilt import create_react_agent
from langchain_core.messages import SystemMessage
```

---

2. Transport 방식 비교: stdio vs Streamable HTTP

항목	stdio	Streamable HTTP
연결 방식	stdin/stdout 파이프	HTTP POST + SSE (Server-Sent Events)
서버 실행	클라이언트가 서버 프로세스 직접 실행	서버가 독립적으로 미리 실행되어 있어야 함
프로세스 생명주기	클라이언트와 함께 시작·종료	서버가 독립적으로 지속 실행
네트워크	불필요 (로컬 IPC)	HTTP 통신 필요
설정 복잡도	낮음 (command/args만 지정)	중간 (URL, 인증 헤더 설정)
보안	높음 (외부 노출 없음)	네트워크 보안 별도 고려 필요
원격 서버	불가	가능 (인터넷/인트라넷 서버 연결)
대표 사용 사례	로컬 도구 (파일시스템, bash, outlook)	원격 API 서버, 공유 서비스

언제 어느 방식을 선택하나?

- **stdio**: 대부분의 로컬 MCP 서버 (npx 패키지, 로컬 Python 스크립트)
- **Streamable HTTP**: 팀 공유 서버, 클라우드 배포 서버, 인증이 필요한 원격 서비스

3. stdio 방식 상세 - 서버를 클라이언트에서 실행하는 이유

왜 클라이언트가 서버를 직접 실행하는가?

stdio 방식의 MCP 서버는 **독립적인 HTTP 서버가 아니라 stdin/stdout을 통신 채널로 사용하는 프로세스**입니다.

```
클라이언트 프로세스
    │
    ├─ stdin  ──→  [MCP 서버 프로세스]  (JSON-RPC 요청 전송)
    └─ stdout ←──  [MCP 서버 프로세스]  (JSON-RPC 응답 수신)
```

MCP 서버 프로세스는 다음과 같은 특징을 가집니다:

- **포트 바인딩 없음**: 네트워크 소켓을 열지 않습니다
- **stdin 대기**: 시작하면 stdin으로 JSON-RPC 메시지를 기다립니다
- **단일 클라이언트**: 하나의 클라이언트(부모 프로세스)와만 통신합니다

따라서 **클라이언트가 서버 프로세스를 자식 프로세스로 직접 생성**해야만 stdin/stdout 파이프가 연결됩니다. 미리 서버를 별도로 실행해 두면 stdin이 터미널에 연결된 상태라 클라이언트와 통신할 수 없습니다.

내부 동작 원리

```python
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@upstash/context7-mcp@latest"],
)

# stdio_client가 내부적으로 subprocess.Popen으로 서버 실행
# 파이프(stdin/stdout)를 통해 JSON-RPC 통신
async with stdio_client(server_params) as (read, write):
    # read: 서버의 stdout → 클라이언트가 읽는 스트림
    # write: 클라이언트 → 서버 stdin으로 쓰는 스트림
    async with ClientSession(read, write) as session:
        await session.initialize()  # MCP 핸드셰이크
        tools = await load_mcp_tools(session)
```

`stdio_client` 컨텍스트 종료 시 자식 프로세스(MCP 서버)도 자동으로 종료됩니다.

4. stdio 서버 실행 방식 - 서버 코드에 따른 command 설정

서버가 어떤 언어/방식으로 작성되었느냐에 따라 `command`와 `args`가 달라집니다.

방식 1: npx — Node.js 패키지 서버

npm에 공개된 MCP 서버를 실행합니다. Node.js와 npm이 설치되어 있어야 합니다.

```
# @upstash/context7-mcp: 라이브러리 문서 조회
server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@upstash/context7-mcp@latest"],
)

# @modelcontextprotocol/server-filesystem: 파일시스템 조작
server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@modelcontextprotocol/server-filesystem", "/allowed/path"],
)

# @modelcontextprotocol/server-sequential-thinking: 단계별 추론
server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@modelcontextprotocol/server-sequential-thinking"],
)
```

- `-y`: 설치 확인 프롬프트 자동 수락
- `@latest`: 항상 최신 버전 사용 (고정하려면 `@1.2.3` 형식)
- 패키지가 없으면 자동 다운로드 후 실행

방식 2: uvx — Python 패키지 서버 (PyPI)

PyPI에 공개된 Python MCP 서버를 임시 가상환경에서 실행합니다. `uv`가 설치되어 있어야 합니다.

```
# mcp-server-fetch: 웹 콘텐츠 가져오기
server_params = StdioServerParameters(
    command="uvx",
    args=["mcp-server-fetch"],
)

# office-powerpoint-mcp-server: PowerPoint 자동화
# 패키지명과 실행 명령어가 다를 때 --from 사용
server_params = StdioServerParameters(
    command="uvx",
    args=["--from", "office-powerpoint-mcp-server", "ppt_mcp_server"],
)
```

- `uvx [패키지명]`: 패키지명 = 실행 명령어가 같을 때
- `uvx --from [패키지명] [실행명령어]`: 패키지명과 실행 명령어가 다를 때

방식 3: uv run — 로컬 Python 서버 (FastMCP 기반)

로컬에 작성한 Python MCP 서버(`server.py`)를 실행할 때 사용합니다. `mcp[cli]` 패키지를 임시로 포함하여 실행합니다.

```
import os

MCP_SERVER_PATH = os.path.join(os.path.dirname(__file__), "mcp-bash", "server.py")

server_params = StdioServerParameters(
    command="uv",
    args=[
        "run",
        "--with", "mcp[cli]",  # mcp CLI 도구를 임시 포함
        "mcp", "run",           # mcp CLI로 서버 실행
        MCP_SERVER_PATH,        # 서버 스크립트 경로
    ],
)
```

이 방식은 서버 스크립트가 **FastMCP** 기반으로 작성된 경우에 적합합니다.

FastMCP 서버 작성 예시 (`server.py`):

```
import subprocess
from typing import Tuple
import os
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Bash")  # 서버 이름 지정

GLOBAL_CWD = os.getcwd()

@mcp.tool()
async def set_cwd(path: str) -> str:
    """작업 디렉터리를 설정합니다."""
    global GLOBAL_CWD
    if not os.path.isdir(path):
        raise ValueError(f"Invalid directory: {path}")
    GLOBAL_CWD = path
    return f"Working directory set to: {GLOBAL_CWD}"

@mcp.tool()
async def execute_bash(cmd: str) -> Tuple[str, str]:
    """bash 명령어를 실행합니다."""
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
        shell=True,
        cwd=GLOBAL_CWD,
    )
```

```
    stdout, stderr = process.communicate()
    return stdout, stderr

# FastMCP는 mcp run으로 실행 시 자동으로 stdio 모드 진입
# if __name__ == "__main__" 불필요
```

방식 4: sys.executable — 현재 Python 인터프리터로 직접 실행

현재 가상환경의 Python으로 특정 스크립트를 직접 실행합니다. 서버 스크립트가 `if __name__ == "__main__": mcp.run()` 형태로 작성된 경우에 사용합니다.

```
import sys
import os

server_script = os.path.join(
    os.path.dirname(__file__), "outlook-mcp-server", "outlook_mcp_server.py"
)

server_params = StdioServerParameters(
    command=sys.executable,   # 현재 Python 인터프리터 경로
    args=[server_script],
    env={**os.environ, "TRANSPORT": "stdio"},  # 환경변수 전달 가능
)
```

서버 스크립트 구조 예시:

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("outlook-assistant")

@mcp.tool()
def list_recent_emails(days: int = 7) -> str:
    """최근 이메일 목록을 반환합니다."""
    ...

if __name__ == "__main__":
    mcp.run()  # stdio 모드로 실행
```

방식별 비교 요약

방식	command	서버 위치	전제 조건
`npx`	`npx`	npm 레지스트리	Node.js 18+, npm
`uvx`	`uvx`	PyPI	uv 설치
`uv run`	`uv`	로컬 파일	uv 설치, FastMCP 기반

방식	command	서버 위치	전제 조건
`sys.executable`	`sys.executable`	로컬 파일	현재 venv에 의존성 설치

5. Streamable HTTP 방식 상세

Streamable HTTP는 이미 실행 중인 HTTP 기반 MCP 서버에 연결합니다.

MultiServerMCPClient에서 설정

```
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        "my_remote_server": {
            "url": "http://localhost:8000/mcp",
            "transport": "streamable_http",
        },
    }
)
tools = await client.get_tools()
```

인증 헤더 추가

```
client = MultiServerMCPClient(
    {
        "secure_server": {
            "url": "https://api.example.com/mcp",
            "transport": "streamable_http",
            "headers": {
                "Authorization": "Bearer YOUR_API_TOKEN",
                "X-API-Key": "your-key",
            },
        },
    }
)
```

mcp 라이브러리로 직접 연결

```
from mcp.client.streamable_http import streamable_http_client
from mcp import ClientSession
from langchain_mcp_adapters.tools import load_mcp_tools

async with streamable_http_client("http://localhost:8000/mcp") as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        tools = await load_mcp_tools(session)
```

서버 측 구현 (FastMCP HTTP 서버)

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-server")

@mcp.tool()
def search(query: str) -> str:
    """검색을 수행합니다."""
    return f"Results for: {query}"

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="0.0.0.0", port=8000)
```

실행:

```
python server.py
# 서버가 http://0.0.0.0:8000/mcp 에서 대기
```

> **핵심**: HTTP 방식은 서버를 **미리 별도로 실행**해두어야 합니다. 클라이언트가 서버를 실행하지 않습니다.

---

6. stdio + HTTP 혼합 사용

`MultiServerMCPClient`는 서버마다 다른 Transport를 설정할 수 있습니다.

```
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        # stdio 방식 — 로컬 파일시스템 서버
        "filesystem": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem",
"/workspace"],
            "transport": "stdio",
        },
        # stdio 방식 — 로컬 bash 서버
        "bash": {
            "command": "uv",
            "args": ["run", "--with", "mcp[cli]", "mcp", "run",
"/path/to/server.py"],
            "transport": "stdio",
        },
        # HTTP 방식 — 원격 검색 서버
        "search": {
```

```
            "url": "https://search-api.internal/mcp",
            "transport": "streamable_http",
            "headers": {"Authorization": "Bearer token123"},
        },
        # HTTP 방식 — 로컬 실행 중인 커스텀 서버
        "custom_api": {
            "url": "http://localhost:8000/mcp",
            "transport": "streamable_http",
        },
    }
)

tools = await client.get_tools()
print(f"총 {len(tools)}개 도구 로드됨")
```

이렇게 하면 stdio 서버들의 프로세스 관리와 HTTP 서버 연결이 동시에 처리됩니다.

---

7. Sync vs Async 방식

langchain-mcp-adapters는 async 전용

`langchain-mcp-adapters`의 핵심 API는 **모두 비동기(async)**입니다:

- `load_mcp_tools(session)` → `await` 필요
- `client.get_tools()` → `await` 필요
- `stdio_client(...)` → `async with` 필요
- `agent.ainvoke(...)`, `agent.astream(...)` → `async for` / `await` 필요

이유: MCP 서버와의 통신은 I/O 바운드 작업이며, stdio 파이프 및 HTTP 통신 모두 비동기 처리가 필수입니다.

일반 Python 스크립트에서 실행

```
import asyncio

async def main():
    # 모든 비동기 코드 작성
    client = MultiServerMCPClient({...})
    tools = await client.get_tools()
    ...

if __name__ == "__main__":
    asyncio.run(main())
```

Jupyter Notebook에서 실행

Jupyter는 이미 이벤트 루프가 실행 중이므로 `asyncio.run()` 대신 `await`를 직접 사용합니다:

```
# Jupyter 셀에서 직접 실행 가능
client = MultiServerMCPClient({...})
tools = await client.get_tools()
print(f"Loaded {len(tools)} tools")
```

동기 코드에서 비동기 함수 호출이 필요한 경우

부득이하게 동기 컨텍스트에서 호출해야 한다면:

```
import asyncio

def sync_wrapper():
    async def _inner():
        client = MultiServerMCPClient({...})
        return await client.get_tools()

    # 새 이벤트 루프 생성 (이미 루프가 있으면 에러 발생)
    return asyncio.run(_inner())
```

> **주의**: 이미 실행 중인 이벤트 루프에서 `asyncio.run()`을 호출하면 오류가 발생합니다. 이 경우 `nest_asyncio` 라이브러리를 사용하거나 전체 코드를 async로 작성하세요.

---

8. Session 관리: Stateless vs Stateful

개념 비교

구분	Stateful	Stateless
세션 생명주기	프로그램 시작 ~ 종료까지 유지	요청마다 생성 후 즉시 종료
서버 프로세스	하나의 프로세스 지속 실행	요청마다 새 프로세스 생성·종료
서버 내부 상태	유지됨 (e.g., 작업 디렉터리, 캐시)	매 요청마다 초기화됨
메모리 사용	서버 프로세스가 상주	요청 처리 중에만 프로세스 존재
적합한 서버	상태가 있는 서버 (bash, outlook)	상태가 없는 서버 (fetch, search)
코드 패턴	`async with` 컨텍스트 전체를 유지	함수 호출마다 `async with`

패턴 1: Stateful — 장기 세션 (프로그램 전체 유지)

세션을 프로그램 시작 시 한 번 열고 종료까지 유지합니다. 서버의 내부 상태(작업 디렉터리, 인증 정보, 캐시 등)가 요청 간에 보존됩니다.

```
import asyncio
from dotenv import load_dotenv
```

```python
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="uv",
    args=["run", "--with", "mcp[cli]", "mcp", "run", "server.py"],
)

async def main():
    load_dotenv()

    # ── 세션을 프로그램 전체 기간 유지 ──
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)

            llm = ChatAnthropic(model="claude-sonnet-4-20250514")
            agent = create_react_agent(llm, tools)

            # ──────────────────────────────────────────
            # 첫 번째 요청: 작업 디렉터리를 /workspace로 설정
            await agent.ainvoke({"messages": [("user", "작업 디렉터리를 /workspace
로 설정해줘")]})

            # 두 번째 요청: 위에서 설정한 /workspace 상태가 서버에 그대로 남아있음
            response = await agent.ainvoke({"messages": [("user", "현재 디렉터리에
서 ls 실행해줘")]})
            # → /workspace 기준으로 ls 실행됨 (상태 유지 확인)
            # ──────────────────────────────────────────

            print(response["messages"][-1].content)
    # async with 블록이 끝나면 세션 종료 + 서버 프로세스 자동 종료

if __name__ == "__main__":
    asyncio.run(main())
```

핵심: `async with stdio_client(...)` 블록과 `async with ClientSession(...)` 블록이 프로그램이 실행되는 동안 계속 열려 있어야 상태가 유지됩니다.

패턴 2: Stateful — 대화 이력 유지 (멀티턴 챗봇)

에이전트 레벨에서 메시지 이력을 누적하면 서버 상태 + 대화 문맥이 함께 유지됩니다.

```python
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
```

```python
from langchain_core.messages import HumanMessage, AIMessage, BaseMessage
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="uv",
    args=["run", "--with", "mcp[cli]", "mcp", "run", "server.py"],
)

async def main():
    load_dotenv()

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)

            llm = ChatAnthropic(model="claude-sonnet-4-20250514")
            agent = create_react_agent(llm, tools)

            # 대화 이력 누적 (Stateful 대화)
            conversation_history: list[BaseMessage] = []

            print("대화를 시작합니다. (종료: quit)\n")
            while True:
                try:
                    user_input = input("You: ").strip()
                except (EOFError, KeyboardInterrupt):
                    break

                if not user_input or user_input.lower() in ("quit", "exit"):
                    break

                # 현재 입력을 이력에 추가
                conversation_history.append(HumanMessage(content=user_input))

                response = await agent.ainvoke(
                    {"messages": conversation_history}
                )

                # 에이전트 응답을 이력에 누적
                ai_reply: BaseMessage = response["messages"][-1]
                conversation_history.append(ai_reply)

                print(f"\nAgent: {ai_reply.content}\n")
                print(f"[대화 이력: {len(conversation_history)}개 메시지]\n")

if __name__ == "__main__":
    asyncio.run(main())
```

> **주의**: 대화 이력이 길어지면 토큰 비용이 증가합니다. 필요에 따라 최근 N개 메시지만 유지하거나 요약하는 전략을 사용하세요.

패턴 3: Stateless — 요청마다 새 세션 (일회성 호출)

각 요청마다 서버 프로세스를 새로 시작하고 종료합니다. 서버 상태가 없는(stateless) 서버에 적합합니다 (예: fetch, 검색, 단순 계산).

```
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="uvx",
    args=["mcp-server-fetch"],
)

async def run_single_query(user_input: str) -> str:
    """요청마다 새 세션을 생성하고 응답 후 즉시 종료합니다."""
    load_dotenv()

    # — 함수 호출마다 세션 생성 → 응답 → 세션 종료 —
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)

            llm = ChatAnthropic(model="claude-sonnet-4-20250514")
            agent = create_react_agent(llm, tools)

            response = await agent.ainvoke(
                {"messages": [("user", user_input)]}
            )
            return response["messages"][-1].content
    # 함수 종료 시 자동으로 세션과 서버 프로세스 정리

async def main():
    # 각 호출이 완전히 독립적
    result1 = await run_single_query("https://example.com 의 제목을 가져와줘")
    print(result1)

    result2 = await run_single_query("https://python.org 의 최신 버전을 알려줘")
    print(result2)
    # result1과 result2 사이에 어떠한 서버 상태도 공유되지 않음

if __name__ == "__main__":
    asyncio.run(main())
```

패턴 4: Stateless — MultiServerMCPClient (자동 세션 관리)

`MultiServerMCPClient`는 내부적으로 각 서버의 세션을 자동 관리합니다. `get_tools()` 호출 시 모든 서버에 연결하고, 이후 도구 호출 시 해당 연결을 재사용합니다.

```python
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

async def main():
    load_dotenv()

    # ✅ 올바른 사용법 (v0.1.0+): 컨텍스트 매니저 없이 인스턴스 생성
    client = MultiServerMCPClient(
        {
            "context7": {
                "command": "npx",
                "args": ["-y", "@upstash/context7-mcp@latest"],
                "transport": "stdio",
            },
            "filesystem": {
                "command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
                "transport": "stdio",
            },
        }
    )

    # get_tools() 호출 시 각 서버에 연결하고 도구 목록을 가져옴
    tools = await client.get_tools()
    # 이후 tools는 프로그램 종료까지 재사용 가능

    llm = ChatAnthropic(model="claude-sonnet-4-20250514")
    agent = create_react_agent(llm, tools)

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() in ("quit", "exit"):
            break
        response = await agent.ainvoke({"messages": [("user", user_input)]})
        print(f"Agent: {response['messages'][-1].content}\n")

if __name__ == "__main__":
    asyncio.run(main())
```

```
# ❌ 잘못된 사용법 — NotImplementedError 발생 (v0.1.0+)
async with MultiServerMCPClient({...}) as client:
    tools = client.get_tools()  # 에러 발생!
```

> **v0.1.0 변경사항**: `MultiServerMCPClient`의 `async with` 지원이 제거되었습니다. 반드시 컨텍스트 매니저 없이 인스턴스를 생성한 후 `await client.get_tools()`를 호출하세요.

패턴 5: Stateless — FastAPI 웹 서버에서 요청별 세션

웹 서버 환경에서는 각 HTTP 요청마다 독립된 MCP 세션을 생성해야 합니다. 요청 간 세션을 공유하면 동시 요청 시 충돌이 발생합니다.

```
import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

load_dotenv()

app = FastAPI()

server_params = StdioServerParameters(
    command="uvx",
    args=["mcp-server-fetch"],
)

class QueryRequest(BaseModel):
    message: str

@app.post("/chat")
async def chat(req: QueryRequest) -> dict:
    # — 각 HTTP 요청마다 완전히 독립된 MCP 세션 생성 —
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)

            llm = ChatAnthropic(model="claude-sonnet-4-20250514")
            agent = create_react_agent(llm, tools)

            response = await agent.ainvoke(
                {"messages": [("user", req.message)]}
            )
```

```
            return {"reply": response["messages"][-1].content}
    # 응답 반환 후 세션 자동 정리
```

> **주의**: 이 패턴은 요청마다 서버 프로세스를 시작·종료하므로 **npx 기반 서버**에서는 매 요청 시 패키지 초기화 비용이 발생합니다. 고트래픽 환경이라면 세션 풀(pool) 또는 장기 세션 패턴을 고려하세요.

패턴 6: Stateful — 장기 세션을 전역으로 관리 (FastAPI 앱 수명주기)

서버 프로세스를 애플리케이션 시작 시 한 번만 실행하고 모든 요청이 공유합니다. 단, 동시 요청 시 세션 충돌에 주의해야 하며 lock을 사용해 직렬화합니다.

```
import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

load_dotenv()

server_params = StdioServerParameters(
    command="uv",
    args=["run", "--with", "mcp[cli]", "mcp", "run", "server.py"],
)

# 전역 공유 상태
_agent = None
_lock = asyncio.Lock()

@asynccontextmanager
async def lifespan(app: FastAPI):
    global _agent
    # ── 앱 시작 시 MCP 세션 1회 초기화 ──
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)
            llm = ChatAnthropic(model="claude-sonnet-4-20250514")
            _agent = create_react_agent(llm, tools)
            print("MCP 세션 초기화 완료")

            yield  # 앱 실행 (이 yield 전후로 startup/shutdown)

    # ── 앱 종료 시 세션 자동 정리 ──
    print("MCP 세션 종료")
```

```
app = FastAPI(lifespan=lifespan)

class QueryRequest(BaseModel):
    message: str

@app.post("/chat")
async def chat(req: QueryRequest) -> dict:
    # Lock으로 동시 요청 직렬화 (하나의 세션을 순차적으로 사용)
    async with _lock:
        response = await _agent.ainvoke(
            {"messages": [("user", req.message)]}
        )
    return {"reply": response["messages"][-1].content}
```

> **트레이드오프**: 세션 초기화 비용 절감 vs 동시 처리 불가 (직렬화). 고동시성 환경에서는 세션 풀(여러 세션 유지)이 더 적합합니다.

패턴 7: HTTP Transport에서의 Stateless

HTTP(Streamable HTTP) 방식의 MCP 서버는 기본적으로 **프로토콜 레벨에서 Stateless**입니다. 각 요청이 독립적인 HTTP 연결을 사용하므로 세션 관리가 단순합니다.

```
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

async def main():
    load_dotenv()

    # HTTP 서버는 미리 실행되어 있어야 함
    # 각 도구 호출은 독립적인 HTTP 요청 → 서버 재시작해도 동일하게 동작
    client = MultiServerMCPClient(
        {
            "remote_search": {
                "url": "http://localhost:8000/mcp",
                "transport": "streamable_http",
                # HTTP는 stateless이므로 세션 상태 걱정 없음
            },
        }
    )

    tools = await client.get_tools()
    llm = ChatAnthropic(model="claude-sonnet-4-20250514")
    agent = create_react_agent(llm, tools)

    response = await agent.ainvoke({"messages": [("user", "검색해줘")]})
    print(response["messages"][-1].content)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

서버 상태 유무에 따른 세션 전략

서버 코드를 보면 어떤 전략을 선택해야 할지 판단할 수 있습니다.

상태가 있는 서버 → Stateful 세션 필수

```
# server.py 예시 — 전역 변수로 상태 유지
GLOBAL_CWD = os.getcwd()   # ← 요청 간에 유지되어야 하는 상태

@mcp.tool()
async def set_cwd(path: str) -> str:
    global GLOBAL_CWD
    GLOBAL_CWD = path       # 다음 요청에도 이 값이 유지되어야 함
    return f"CWD: {GLOBAL_CWD}"

@mcp.tool()
async def execute_bash(cmd: str) -> str:
    # GLOBAL_CWD가 set_cwd로 설정된 값을 사용함
    subprocess.run(cmd, cwd=GLOBAL_CWD, ...)
```

→ `set_cwd` 후 `execute_bash`가 같은 세션에서 실행되어야 합니다. → Stateless(요청마다 새 세션)로 쓰면 `set_cwd`가 의미 없어집니다.

상태가 없는 서버 → Stateless 세션 가능

```
# server.py 예시 — 전역 상태 없음
@mcp.tool()
async def fetch_url(url: str) -> str:
    import httpx
    async with httpx.AsyncClient() as client:
        response = await client.get(url)   # 입력만으로 결과가 결정됨
        return response.text               # 이전 요청에 영향받지 않음
```

→ 매 요청이 완전히 독립적이므로 Stateless 세션이 안전합니다.

세션 선택 기준 요약

상황	권장 패턴	이유
단일 서버, 상태 있음 (bash, outlook)	패턴 1: 장기 Stateful	서버 상태(CWD, 인증 등) 보존 필요
단일 서버, 상태 없음 (fetch, search)	패턴 3: Stateless	매 요청이 독립적, 상태 불필요

상황	권장 패턴	이유
2개 이상 서버	패턴 4: MultiServerMCPClient	다중 서버 자동 관리
멀티턴 대화 필요	패턴 2: 대화 이력 유지	이전 응답 문맥 참조 필요
FastAPI (저트래픽)	패턴 5: 요청별 세션	구현 단순, 격리 보장
FastAPI (고트래픽, 상태 서버)	패턴 6: 앱 수명주기	초기화 비용 절감, lock 직렬화
원격 HTTP 서버	패턴 7: HTTP Stateless	서버가 기본적으로 stateless

9. LangChain Agent와 결합하기

방식 1: create_react_agent (LangGraph, 권장)

ReAct(Reason + Act) 패턴 에이전트로, 도구 호출과 추론을 반복하며 문제를 해결합니다.

```
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import SystemMessage

llm = ChatAnthropic(model="claude-sonnet-4-20250514")

# SystemMessage로 에이전트 행동 제어
agent = create_react_agent(
    llm,
    tools,
    prompt=SystemMessage(content="당신은 파일시스템 전문가입니다. 사용자의 파일 관련 요청을 처리합니다."),
)

# 단순 호출 (스트리밍 없음)
response = await agent.ainvoke(
    {"messages": [("user", "현재 디렉터리의 파일 목록을 보여줘")]}
)
print(response["messages"][-1].content)
```

방식 2: create_agent (langchain.agents)

`langchain.agents`의 기본 에이전트입니다.

```
from langchain.agents import create_agent
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-20250514")
agent = create_agent(llm, tools)

response = await agent.ainvoke(
```

```
    {"messages": [("user", "pandas DataFrame.groupby() 사용법 알려줘")]}
)
```

create_react_agent vs create_agent 비교

항목	`create_react_agent` (LangGraph)	`create_agent` (langchain)
스트리밍	`astream(stream_mode="messages")` 완전 지원	제한적
System Prompt	`prompt=SystemMessage(...)` 직접 지정	별도 설정 필요
도구 라우팅 제어	System Prompt로 세밀하게 제어 가능	제한적
권장 용도	복잡한 멀티 도구 에이전트	단순한 단일 도구 에이전트

System Prompt로 다중 서버 도구 라우팅 제어

여러 MCP 서버를 사용할 때 LLM이 올바른 도구를 선택하도록 System Prompt에서 명시합니다.

```
SYSTEM_PROMPT = """당신은 다목적 AI 어시스턴트입니다. 아래 도구 카테고리를 상황에 맞게
선택하세요.

## 도구 카테고리

### context7 도구 (접두어: context7__)
- 라이브러리/API/프레임워크 문서 조회
- "X 라이브러리 사용법", "Y 함수의 파라미터" 등

### filesystem 도구 (접두어: filesystem__)
- 파일 읽기/쓰기/목록 조회/검색
- "파일을 열어줘", "디렉터리 구조를 보여줘" 등

### sequential_thinking 도구 (접두어: sequential_thinking__)
- 복잡한 분석, 단계적 추론, 설계 결정
- "장단점을 비교해줘", "아키텍처를 설계해줘" 등

## 복합 사용 전략
복잡한 질문에는 여러 도구를 조합하세요:
- 파일 읽기 + 라이브러리 문서 조회: filesystem__ + context7__
- 파일 읽기 + 분석: filesystem__ + sequential_thinking__

도구 사용 전에 어떤 도구를 왜 사용할지 간략히 설명하세요.
"""

agent = create_react_agent(
    llm,
    tools,
    prompt=SystemMessage(content=SYSTEM_PROMPT),
)
```

---

10. AI 답변 실시간 스트리밍

ainvoke vs astream

방식	동작	사용 시기
`ainvoke`	모든 처리가 끝난 후 전체 응답 반환	짧은 응답, 결과만 필요할 때
`astream`	토큰 생성 즉시 실시간으로 스트림	긴 응답, 실시간 출력이 필요할 때

기본 스트리밍 코드

```
async for chunk, metadata in agent.astream(
    {"messages": [("user", user_input)]},
    stream_mode="messages",
):
    if hasattr(chunk, "content") and chunk.content:
        if isinstance(chunk.content, str):
            # 단순 문자열 응답
            print(chunk.content, end="", flush=True)
        elif isinstance(chunk.content, list):
            # Claude의 구조화된 응답 (텍스트 + 도구 호출 블록 혼재)
            for block in chunk.content:
                if isinstance(block, dict):
                    if block.get("type") == "text":
                        # 텍스트 블록
                        print(block["text"], end="", flush=True)
                    elif block.get("type") == "tool_use":
                        # 도구 호출 블록 — 어떤 도구를 쓰는지 실시간 표시
                        print(f"\n[Tool: {block['name']}]", flush=True)
print("\n")  # 마지막 줄바꿈
```

chunk.content 타입 구조 이해

Claude 모델의 응답은 `content`가 두 가지 형태로 올 수 있습니다:

```
# 형태 1: 단순 문자열
chunk.content = "파일 목록을 가져오겠습니다."

# 형태 2: 블록 리스트 (텍스트 + 도구 호출 혼재)
chunk.content = [
    {"type": "text", "text": "filesystem 도구로 파일 목록을 조회합니다.\n"},
    {"type": "tool_use", "id": "tool_abc123", "name":
"filesystem__list_directory", "input": {"path": "."}},
]
```

도구 호출 결과(ToolMessage)는 어떻게 처리하나?

```
from langchain_core.messages import AIMessage, ToolMessage, HumanMessage

async for chunk, metadata in agent.astream(
    {"messages": [("user", user_input)]},
    stream_mode="messages",
):
    # AIMessage: LLM의 응답 (텍스트 + 도구 호출 의도)
    if isinstance(chunk, AIMessage):
        if isinstance(chunk.content, str) and chunk.content:
            print(chunk.content, end="", flush=True)
        elif isinstance(chunk.content, list):
            for block in chunk.content:
                if isinstance(block, dict):
                    if block.get("type") == "text":
                        print(block["text"], end="", flush=True)
                    elif block.get("type") == "tool_use":
                        print(f"\n  → 도구 호출: {block['name']}", flush=True)

    # ToolMessage: 도구 실행 결과 (스킵하거나 디버그용으로 출력)
    elif isinstance(chunk, ToolMessage):
        pass  # 도구 결과는 LLM이 알아서 처리
```

완전한 스트리밍 대화 루프 예제

```
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

async def main():
    load_dotenv()

    client = MultiServerMCPClient(
        {
            "filesystem": {
                "command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
                "transport": "stdio",
            },
        }
    )
    tools = await client.get_tools()

    llm = ChatAnthropic(model="claude-sonnet-4-20250514")
    agent = create_react_agent(llm, tools)

    while True:
        try:
            user_input = input("You: ").strip()
```

```
        except (EOFError, KeyboardInterrupt):
            print("\nExiting.")
            break

        if not user_input:
            continue
        if user_input.lower() in ("quit", "exit", "q"):
            break

        print("\nAgent: ", end="", flush=True)
        try:
            async for chunk, metadata in agent.astream(
                {"messages": [("user", user_input)]},
                stream_mode="messages",
            ):
                if hasattr(chunk, "content") and chunk.content:
                    if isinstance(chunk.content, str):
                        print(chunk.content, end="", flush=True)
                    elif isinstance(chunk.content, list):
                        for block in chunk.content:
                            if isinstance(block, dict):
                                if block.get("type") == "text":
                                    print(block["text"], end="", flush=True)
                                elif block.get("type") == "tool_use":
                                    print(f"\n[Tool: {block['name']}]",
flush=True)
            print("\n")
        except Exception as e:
            print(f"\nError: {e}\n")

if __name__ == "__main__":
    asyncio.run(main())
```

---

11. 여러 MCP 서버 동시 사용 시 주의사항

주의사항 1: 도구 이름 충돌 방지 (tool_prefix)

여러 MCP 서버에 동일한 이름의 도구(예: `list_resources`, `search`)가 존재할 수 있습니다. `MultiServerMCPClient`는 **서버명을 접두어로 자동 추가**하여 충돌을 방지합니다.

```
context7__resolve-library-id
context7__get-library-docs
sequential_thinking__sequentialthinking
filesystem__read_file
filesystem__write_file
filesystem__list_directory
```

접두어 규칙: `{서버 딕셔너리의 키}__{원래 도구 이름}`

```
# 서버 키가 "my_server" → 도구명이 "my_server__search"로 변환됨
client = MultiServerMCPClient({
    "my_server": {  # ← 이 키가 접두어로 사용됨
        "command": "...",
        "transport": "stdio",
    }
})
```

> **주의**: 접두어가 없으면 나중에 로드된 서버의 도구가 이전 서버의 동명 도구를 **조용히 덮어씁니다**. System Prompt에서 접두어 기반 라우팅 규칙을 LLM에게 명시해야 정확하게 동작합니다.

주의사항 2: 로드된 도구 목록 확인

```
tools = await client.get_tools()

# 서버별로 그룹화하여 출력
tool_groups: dict[str, list[str]] = {}
for tool in tools:
    prefix = tool.name.split("__")[0] if "__" in tool.name else "other"
    tool_groups.setdefault(prefix, []).append(tool.name)

print(f"총 {len(tools)}개 도구, {len(tool_groups)}개 서버:")
for server, names in tool_groups.items():
    print(f"  [{server}] {', '.join(names)}")
```

출력 예:

```
총 8개 도구, 3개 서버:
  [context7] context7__resolve-library-id, context7__get-library-docs
  [sequential_thinking] sequential_thinking__sequentialthinking
  [filesystem] filesystem__read_file, filesystem__write_file,
filesystem__list_directory, filesystem__search_files, filesystem__get_file_info
```

주의사항 3: 서버 초기화 실패 처리

`get_tools()` 호출 시 하나의 서버가 실패하면 전체가 실패할 수 있습니다.

```
try:
    tools = await client.get_tools()
    if not tools:
        print("경고: 도구가 0개 로드됨 — 서버 연결 확인 필요")
except Exception as e:
    print(f"MCP 서버 연결 실패: {e}")
    raise
```

일반적인 실패 원인:

- `npx: command not found` → Node.js 미설치
- `uvx: command not found` → uv 미설치
- 서버 스크립트 경로 오류
- 네트워크 연결 문제 (HTTP 방식)

주의사항 4: 서버별 환경변수 설정

일부 MCP 서버는 API 키 등 환경변수가 필요합니다.

```
import os

client = MultiServerMCPClient(
    {
        "my_server": {
            "command": "npx",
            "args": ["-y", "some-mcp-server"],
            "transport": "stdio",
            "env": {
                **os.environ,                       # 현재 환경변수 상속
                "SERVER_API_KEY": "your-key",       # 추가 환경변수
            },
        },
    }
)
```

주의사항 5: 인터셉터(Interceptor) 활용

MCP 클라이언트 레벨에서 요청/응답을 가로채 로깅, 인증 갱신 등을 처리할 수 있습니다.

```
# 개념적 예시 — 실제 API는 라이브러리 버전에 따라 다를 수 있음
# mcp 라이브러리의 ClientSession 사용 시 커스텀 처리 가능

async with ClientSession(read, write) as session:
    await session.initialize()

    # 도구 호출 전/후 로깅 래퍼
    original_tools = await load_mcp_tools(session)

    wrapped_tools = []
    for tool in original_tools:
        original_func = tool.func

        async def logged_func(*args, _tool=tool, _orig=original_func, **kwargs):
            print(f"[LOG] 도구 호출: {_tool.name}, args={args}, kwargs={kwargs}")
            result = await _orig(*args, **kwargs)
            print(f"[LOG] 도구 결과: {_tool.name} → {str(result)[:100]}")
            return result
```

```
        tool.func = logged_func
        wrapped_tools.append(tool)

    agent = create_react_agent(llm, wrapped_tools)
```

주의사항 6: 동시성 — 서버들은 병렬로 초기화됨

`MultiServerMCPClient.get_tools()`는 내부적으로 여러 서버를 **병렬로 초기화**합니다. 서버 초기화에 시간이 걸리는 경우 (예: npx 패키지 다운로드) 첫 실행 시 지연이 있을 수 있습니다.

```
import time

start = time.time()
print("MCP 서버 연결 중...")
tools = await client.get_tools()
print(f"초기화 완료: {time.time() - start:.1f}초, {len(tools)}개 도구 로드됨")
```

---

12. 전체 구조 요약

아키텍처 다이어그램

```
사용자 입력
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│                    Python 클라이언트                      │
│                                                          │
│  ┌──────────────────────────────────────────────┐        │
│  │          MultiServerMCPClient                 │        │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐   │        │
│  │  │ server1  │  │ server2  │  │ server3  │   │        │
│  │  │ (stdio)  │  │ (stdio)  │  │  (HTTP)  │   │        │
│  │  └──────────┘  └──────────┘  └──────────┘   │        │
│  └──────────┬──────────────┬──────────────┬──────┘        │
│             │              │              │               │
│  ┌──────────▼──────────────▼──────────────▼──────┐        │
│  │            tools = [tool1, tool2, ...]        │        │
│  └──────────────────────────┬────────────────────┘        │
│                             │                             │
│  ┌──────────────────────────▼────────────────────┐        │
│  │    create_react_agent(llm, tools)             │        │
│  │    + SystemMessage(라우팅 규칙)               │        │
│  └──────────────────────────┬────────────────────┘        │
│                             │                             │
│  ┌──────────────────────────▼────────────────────┐        │
│  │    agent.astream(stream_mode="messages")      │        │
│  │    → 실시간 토큰 출력 + 도구 호출 표시        │        │
│  └──────────────────────────────────────────────┘        │
```

```
└──────────────────────────────────────────────────────┘
          │               │            │
          ▼               ▼            ▼
  [MCP 서버 프로세스]  [MCP 서버 프로세스]  [HTTP MCP 서버]
  (자식 프로세스)      (자식 프로세스)      (원격 서버)
  stdin/stdout       stdin/stdout       HTTP POST+SSE
```

상황별 패턴 선택 가이드

상황	권장 패턴	핵심 코드
단일 공개 MCP 서버 (npx/uvx)	`stdio_client` + `load_mcp_tools`	`StdioServerParameters(command="npx", ...)`
단일 로컬 Python 서버	`stdio_client` + `sys.executable`	`StdioServerParameters(command=sys.executable, ...)`
단일 로컬 FastMCP 서버	`stdio_client` + `uv run`	`StdioServerParameters(command="uv", args=["run", "--with", "mcp[cli]", "mcp", "run", ...])`
2개 이상 서버	`MultiServerMCPClient`	`client = MultiServerMCPClient({...}); tools = await client.get_tools()`
원격 HTTP 서버	`MultiServerMCPClient` (HTTP)	`{"transport": "streamable_http", "url": "..."}`
실시간 스트리밍	`astream` + `stream_mode`	`agent.astream({...}, stream_mode="messages")`
도구 충돌 방지	서버 키 명명	딕셔너리 키 = 접두어로 사용됨

최소 동작 코드 (빠른 시작)

```python
import asyncio
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

async def main():
    load_dotenv()  # ANTHROPIC_API_KEY 로드

    client = MultiServerMCPClient({
        "fetch": {
            "command": "uvx",
            "args": ["mcp-server-fetch"],
            "transport": "stdio",
        },
    })
    tools = await client.get_tools()
```

```
    agent = create_react_agent(ChatAnthropic(model="claude-sonnet-4-20250514"),
tools)

    response = await agent.ainvoke({"messages": [("user", "https://example.com 의
내용을 가져와줘")]})
    print(response["messages"][-1].content)

if __name__ == "__main__":
    asyncio.run(main())
```

자주 발생하는 오류와 해결책

오류	원인	해결
`NotImplementedError: MultiServerMCPClient`	`async with` 사용	`client = MultiServerMCPClient({...})` 후 `await client.get_tools()`
`npx: command not found`	Node.js 미설치	Node.js 18+ 설치
`uvx: command not found`	uv 미설치	`pip install uv` 또는 공식 설치
도구 0개 로드됨	서버 연결 실패	서버 command/args 확인, 인터넷 연결 확인
`ModuleNotFoundError: langchain_mcp_adapters`	패키지 미설치	`pip install langchain-mcp-adapters`
`ANTHROPIC_API_KEY` 오류	환경변수 미설정	`.env` 파일에 `ANTHROPIC_API_KEY=sk-ant-...` 추가
파일 접근 거부 (filesystem)	허용 경로 외 접근	`args`에 허용 디렉터리 경로 추가
HTTP 연결 오류	서버 미실행	HTTP 방식은 서버를 미리 별도 실행 필요

작성 기준: langchain-mcp-adapters v0.1.x, mcp v1.x, langgraph v0.2.x